

Soluciones Directas para Sistemas Lineales

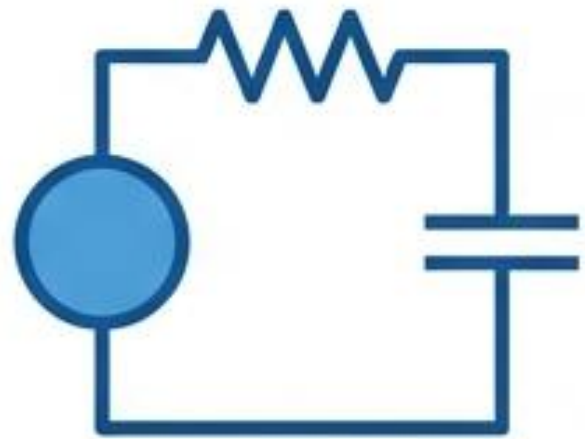
Desde la Aritmética del Computador
hasta la Factorización Avanzada

Curso: Métodos Numéricos
Docente: Dr. Hermes Pantoja Carhuavilca



Motivación: La Ubicuidad de $Ax = b$ en Ingeniería

Los métodos analíticos funcionan para 3 variables; los numéricos para 3 millones.



Circuitos Eléctricos

Leyes de Kirchhoff.
Balance de corrientes
y voltajes.



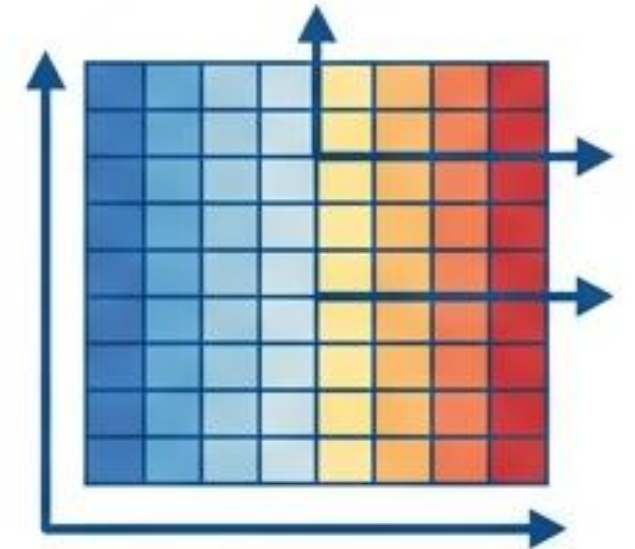
Estructuras

Análisis de armaduras
y elementos finitos
(Matriz de rigidez).



Ingeniería Química

Balance de masa en
reactores
interconectados.



Modelado Térmico

Ecuaciones
discretizadas en mallas
(Diferencias finitas).

Los métodos directos buscan una solución exacta en un número finito de pasos aritméticos, sirviendo de base para algoritmos iterativos.

Objetivos de la Sesión

01

Implementar

Algoritmos de Eliminación Gaussiana y Factorización LU para la resolución de sistemas matriciales de gran escala.

02

Evaluar

El número de condicionamiento ($\text{cond}(A)$) para predecir la sensibilidad de la solución ante perturbaciones en los datos de entrada.

03

Validar

La estabilidad numérica mediante estrategias de pivoteo parcial, minimizando el error de redondeo inherente a la aritmética de punto flotante.

04

Analizar

Sistemas físicos reales (microrredes eléctricas y balances de flujo) traduciéndolos a modelos matriciales del tipo $Ax = b$.

Anatomía del Sistema: Forma Algebraica y Matricial

The Math
(Representación Algebraica)

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m \end{cases}$$

The Data Structure
(Representación Matricial)

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

Matriz de
Coeficientes
(A): $m \times n$

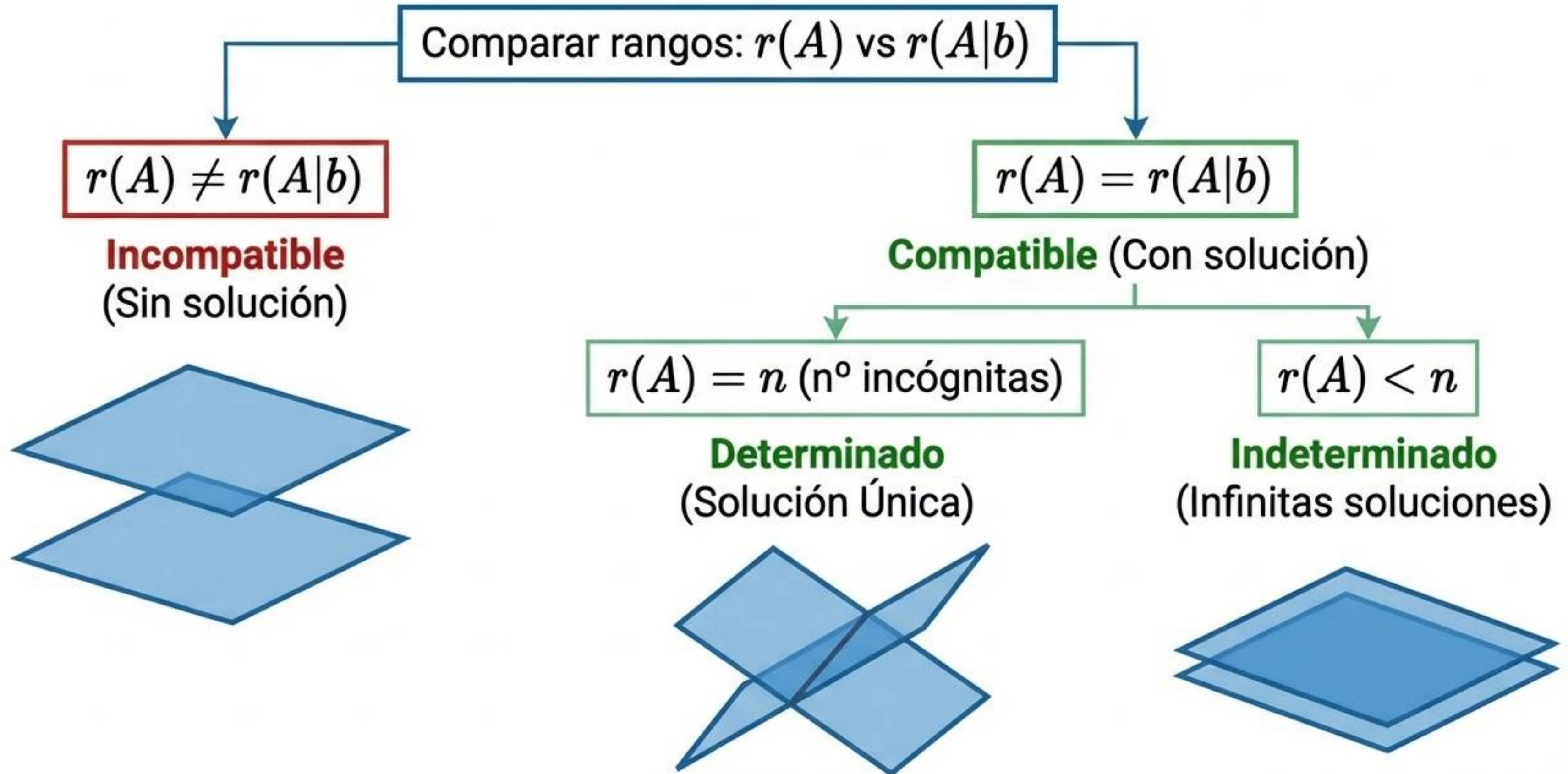
Vector
Incógnita
(x): $n \times 1$

Vector Términos
Independientes
(b): $m \times 1$

Matriz Aumentada
(B o $[A|b]$)

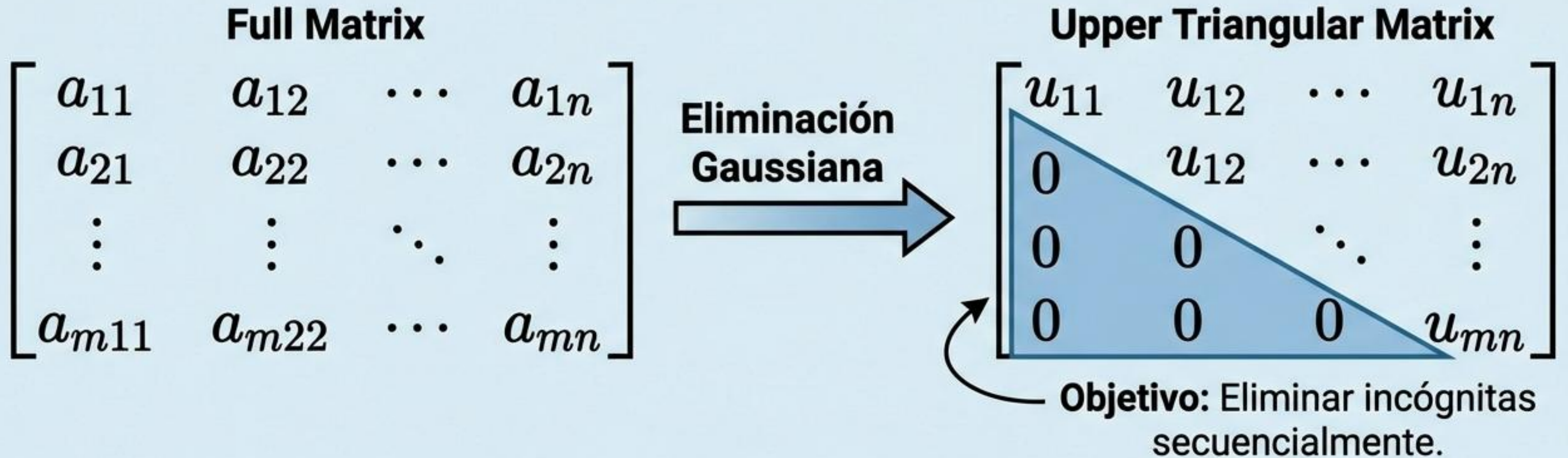
$$\left[\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{array} \right]$$

Existencia y Unicidad: Teorema de Rouché–Frobenius



Estrategia de Solución: Triangularización

Transformar $\mathbf{Ax}=\mathbf{b}$ en $\mathbf{Ux}=\mathbf{c}$ mediante Operaciones Elementales



3 Operaciones Legales

1. **Intercambio** ($R_i \leftrightarrow R_j$): Crucial para el pivoteo.
2. **Escalamiento** ($R_i \rightarrow kR_i$): Multiplicar una fila por un escalar no nulo.
3. **Eliminación** ($R_i \rightarrow R_i - \lambda R_j$): Sumar un múltiplo de una fila a otra.

Algoritmo Base: Eliminación Gaussiana (Naive)

Fase 1: Eliminación Hacia Adelante

Objetivo: Hacer ceros debajo de la diagonal principal.

Multiplicador: $m_{ik} = a_{ik}/a_{kk}$

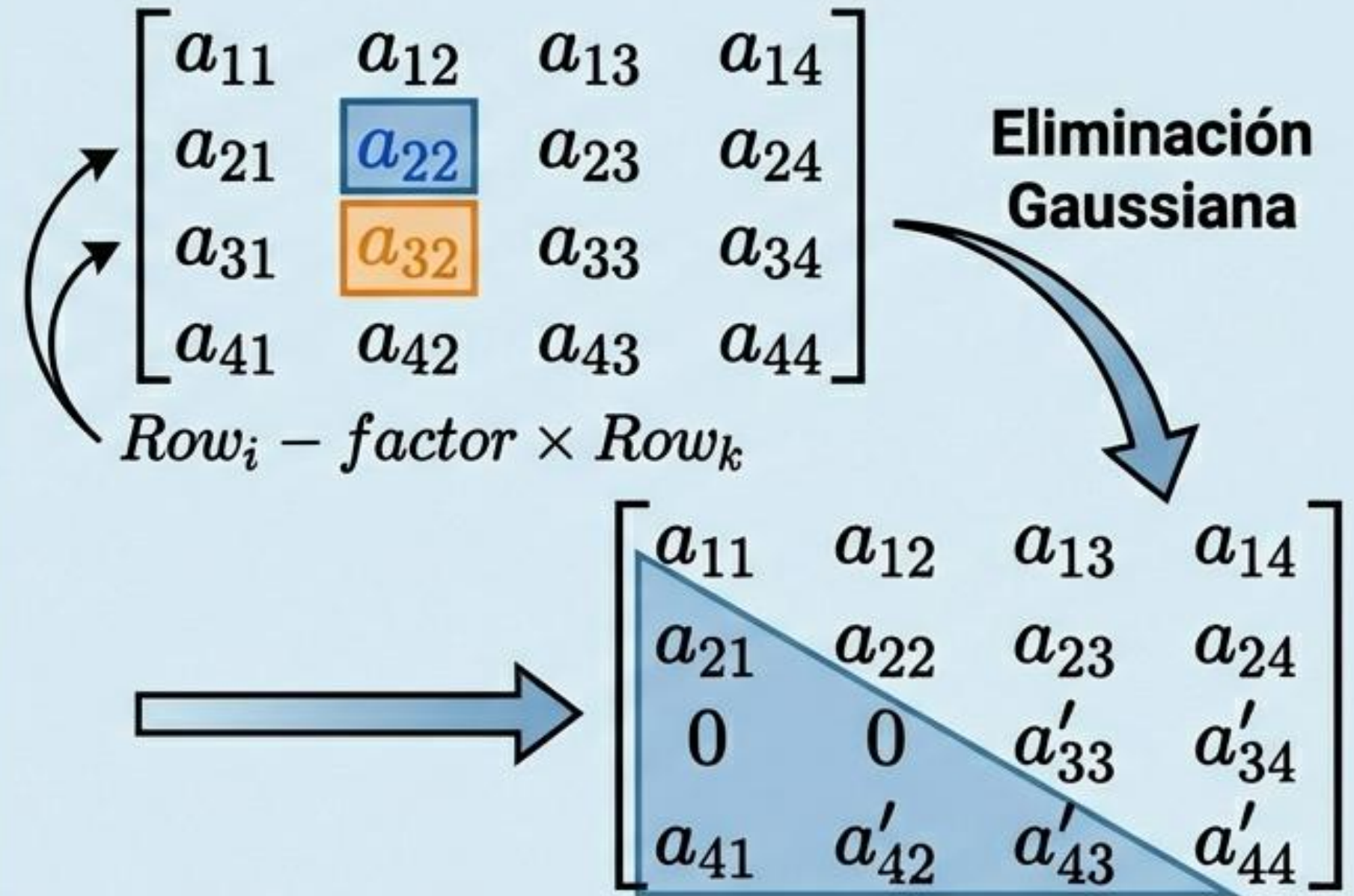
Para $k = 1$ hasta $n-1$:

Para $i = k+1$ hasta n :

factor = $A[i,k] / A[k,k]$

$A[i,:] = A[i,:] - \text{factor} * A[k,:]$

The Visualization



Objetivo: Hacer ceros debajo de la diagonal principal.

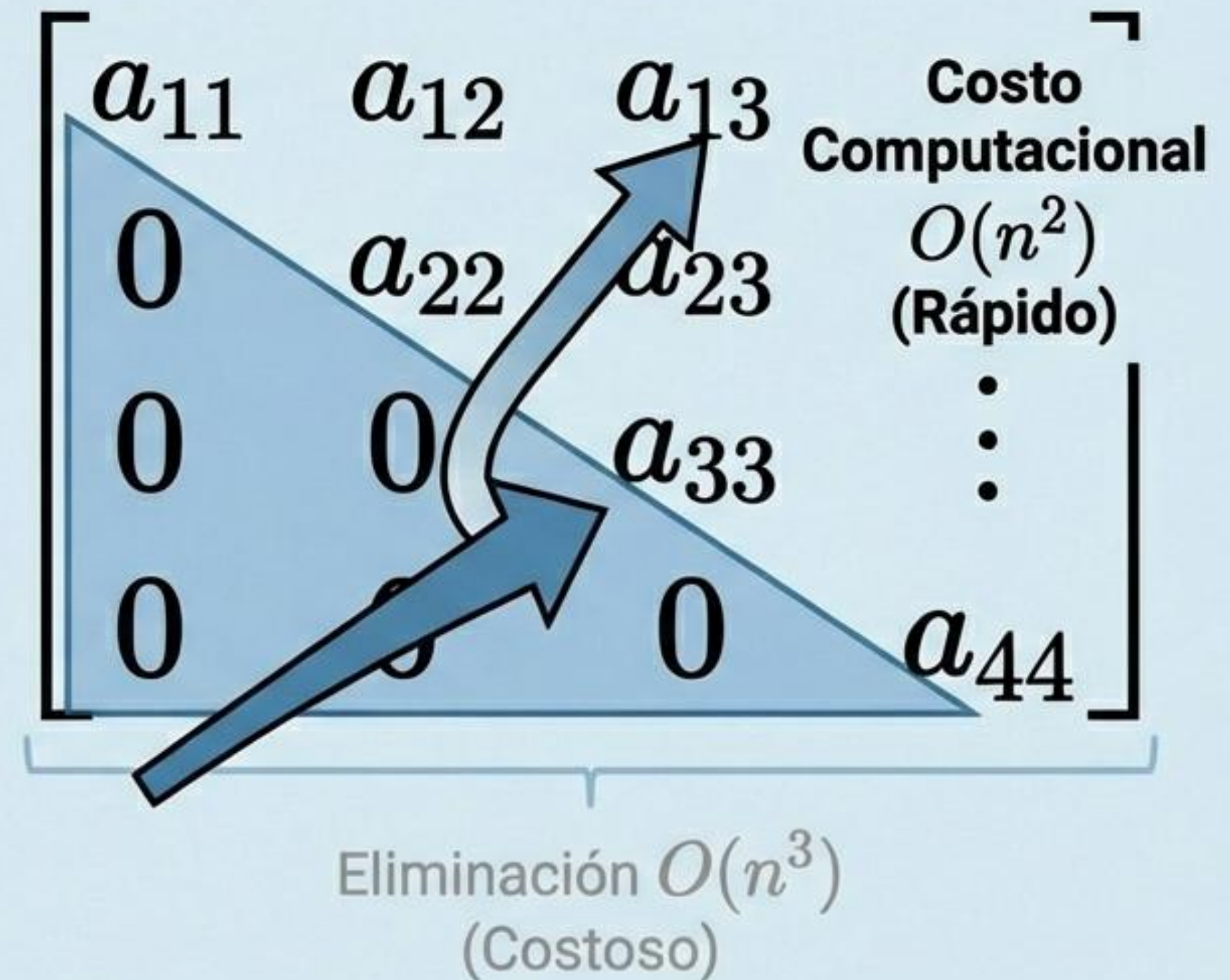
Resolución: Sustitución Hacia Atrás (Back Substitution)

Fase 2: Despeje Recursivo

Se resuelve desde la última ecuación (x_n) hacia la primera (x_1).

$$x_i = \frac{b_i - \sum_{j=i+1}^n a_{ij}x_j}{a_{ii}}$$

The Visualization



Fragilidad Numérica: El Problema de los Pivotes Pequeños

El Escenario de Falla

Pivote $a_{kk} \approx 0$ (ej. 0.000001) ⚠

→ Multiplicador $m = \text{Grande} / 0.000001 = \text{Enorme}$

→ El **error** de redondeo se **multiplica** por **"Enorme"**

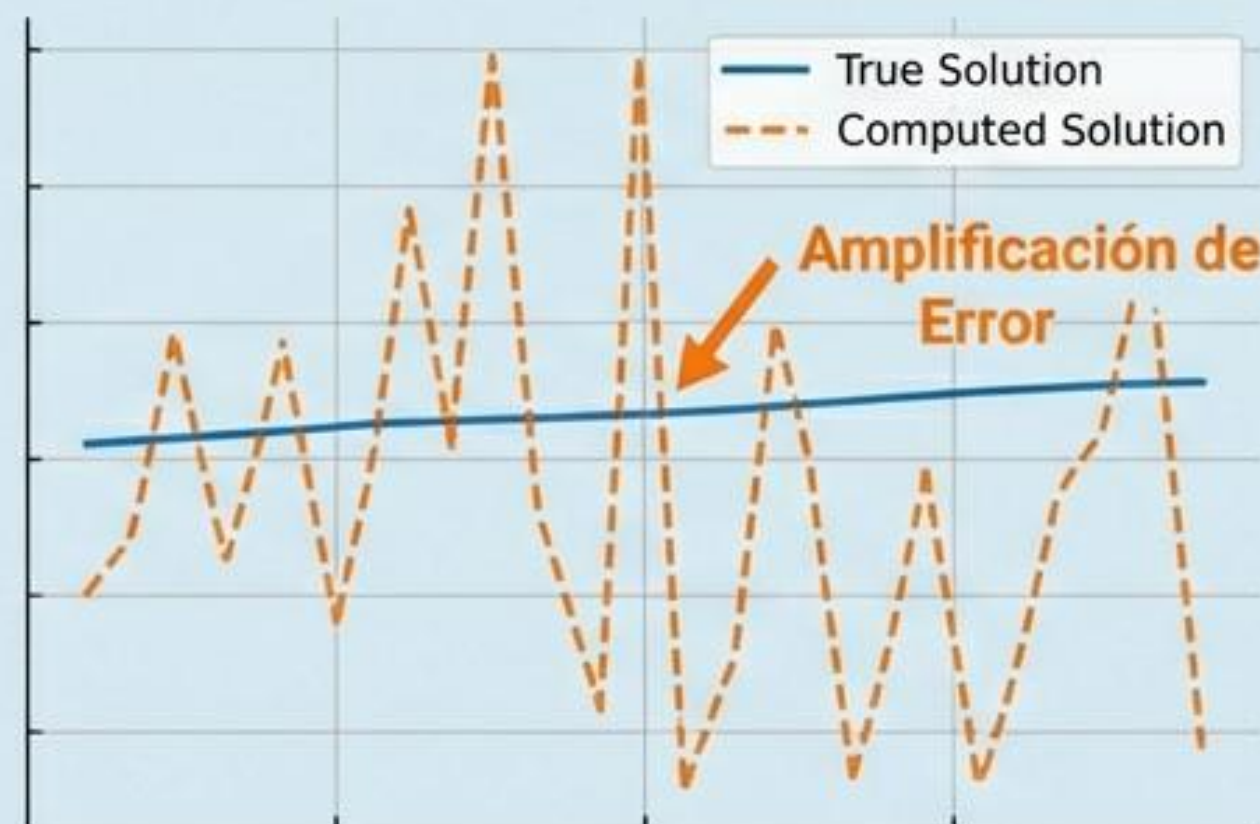
Consecuencias



Amplificación de Errores: La precisión finita del computador destruye la solución.



División por Cero: Si $a_{kk} = 0$, el algoritmo se detiene.



Eliminación Gaussiana: El Riesgo del Pivote Pequeño

El Algoritmo: Transformación de A en una matriz triangular superior U (Forward Elimination).

El problema del error de redondeo: La fórmula de eliminación requiere dividir por el elemento pivote:

$$m_{ik} = \frac{a_{ik}}{a_{kk}}$$

CRÍTICO: Si $a_{kk} \approx 0$, el multiplicador $m_{ik} \rightarrow \infty$, amplificando drásticamente los errores.

 Pivote Pequeño: Causa destrucción de precisión sin intercambio de filas.

$$0.0003x_1 + 3.0000x_2 = 2.0001$$
$$1.0000x_1 + 1.0000x_2 = 1.0000$$

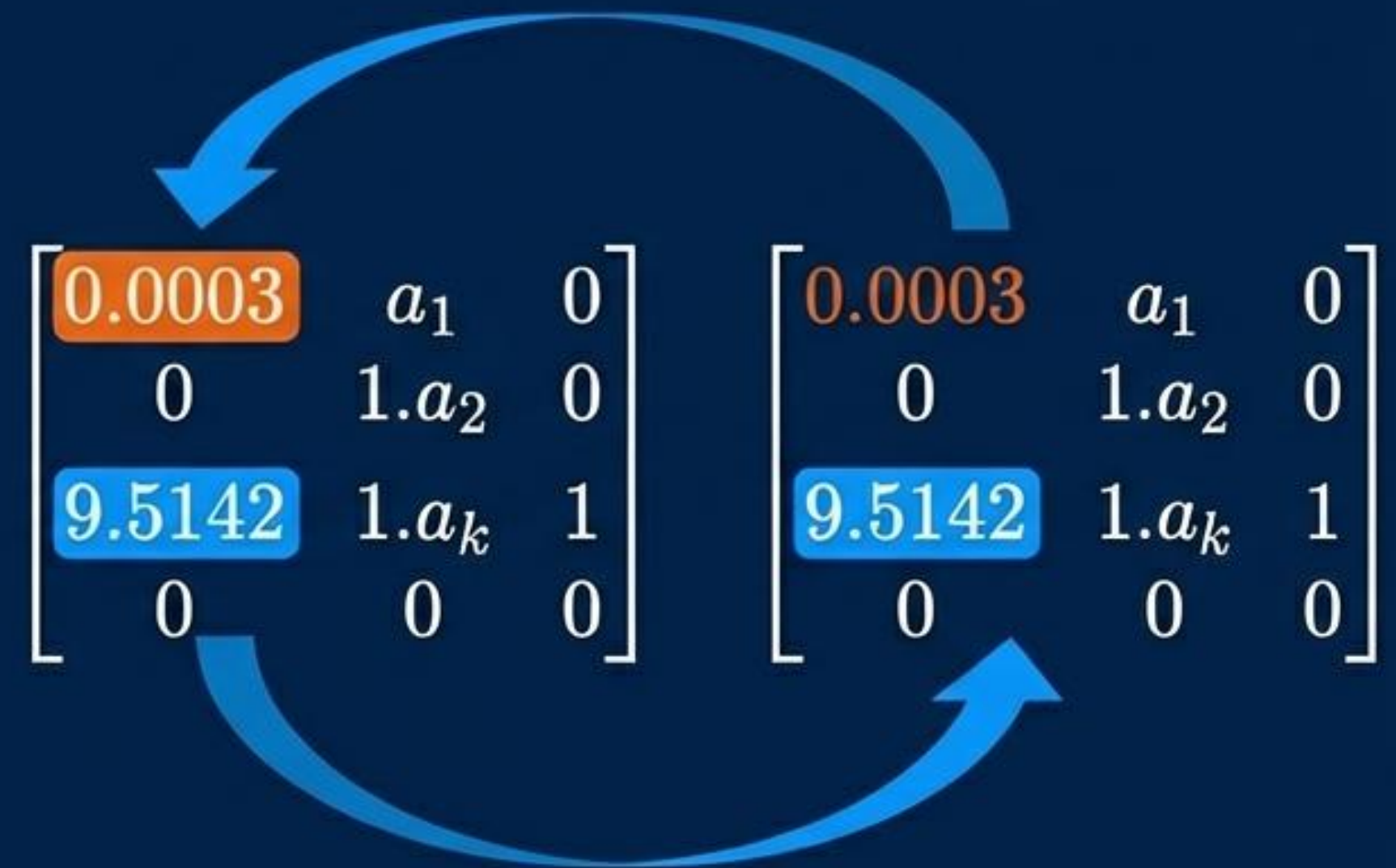
Estrategias de Estabilidad: El Pivoteo

Definición: Intercambio de filas para colocar el coeficiente de mayor magnitud en la posición del pivote.

Pivoteo Parcial (Estándar Industrial):

1. En la etapa k , buscar p tal que:
 $|a_{pk}| = \max |a_{ik}|$ para $k \leq i \leq n$
2. Intercambiar fila $k \leftrightarrow$ fila p .

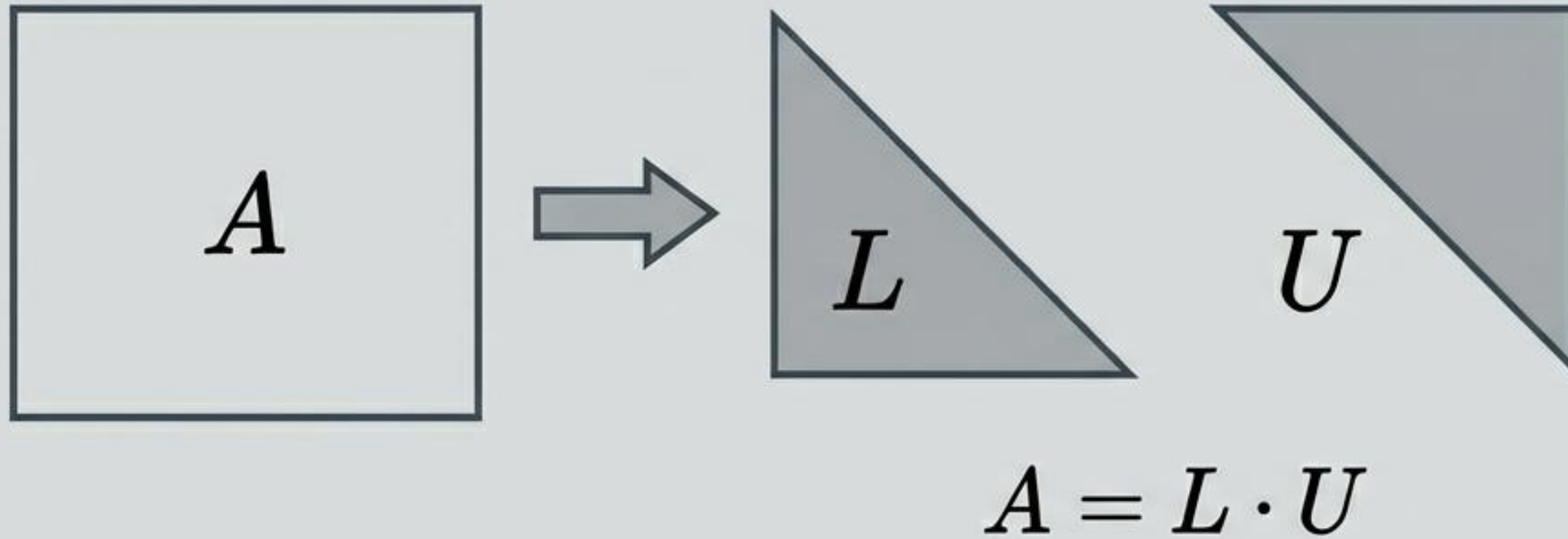
Ventaja: Garantiza multiplicadores $|m_{ik}| \leq 1$, acotando el error.



Intercambio estratégico de filas

Pivoteo Completo: Busca máximo en filas y columnas (Costoso y raramente usado).

Factorización LU: Eficiencia para Múltiples Escenarios



Flujo de Trabajo:

1. Descomposición: $A \rightarrow L, U$ (Costo $O(n^3/3)$)
2. Sustitución hacia adelante: $Ly = b$ (Hallar y)
3. Sustitución hacia atrás: $Ux = y$ (Hallar x)

Variantes:

Método de Doolittle:

$l_{ii} = 1$
(1s en diagonal de L)

Método de Crout:

$u_{ii} = 1$
(1s en diagonal de U)

Método de Cholesky:

$A = LL^T$
(Para matrices simétricas
positivas definidas)

Variantes LU: Métodos de Doolittle y Crout

Método de Doolittle

$$\begin{matrix} & L & & U \\ \begin{bmatrix} \boxed{1} & & & \\ l_{21} & \boxed{1} & & \\ l_{31} & l_{32} & \boxed{1} & \\ l_{41} & l_{42} & l_{43} & \boxed{1} \end{bmatrix} & \begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ & u_{22} & u_{23} & u_{24} \\ & & u_{33} & u_{34} \\ & & & u_{44} \end{bmatrix} \end{matrix}$$

$l_{ii} = 1$. Natural de la eliminación Gaussiana.

Método de Crout

$$\begin{matrix} & L & & U \\ \begin{bmatrix} l_{11} & & & \\ l_{21} & l_{22} & & \\ l_{31} & l_{32} & l_{33} & \\ l_{41} & l_{42} & l_{43} & l_{44} \end{bmatrix} & \begin{bmatrix} \boxed{1} & u_{12} & u_{13} & u_{14} \\ & \boxed{1} & u_{23} & u_{24} \\ & & \boxed{1} & u_{34} \\ & & & \boxed{1} \end{bmatrix} \end{matrix}$$

$u_{ii} = 1$.

Ambos métodos requieren $O(n^3/3)$ operaciones. La elección depende de la implementación.

Factorización de Cholesky: Simetría y Velocidad

Requisitos

1. **Simétrica:** $A = A^T$ 
(Espejo respecto a la diagonal).
2. **Definida Positiva:** $x^T A x > 0$
(Energía positiva).

$$A = L \cdot L^T$$

$$\begin{matrix} & A & & L & & L^T \\ \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{24} \\ a_{31} & a_{32} & a_{33} \\ a_{41} & a_{42} & a_{44} \end{bmatrix} & = & \begin{bmatrix} 0 & 0 & 0 \\ l_{11} & 0 & 0 \\ l_{31} & l_{32} & 0 \\ l_{41} & l_{42} & l_{k3} \end{bmatrix} & \times & \begin{bmatrix} l_{ji} & l_{j1} & l_{14} \\ 0 & l_{22} & l_{21} \\ 0 & 0 & l_{34} \\ 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

 $l_{kk} = \sqrt{\dots}$

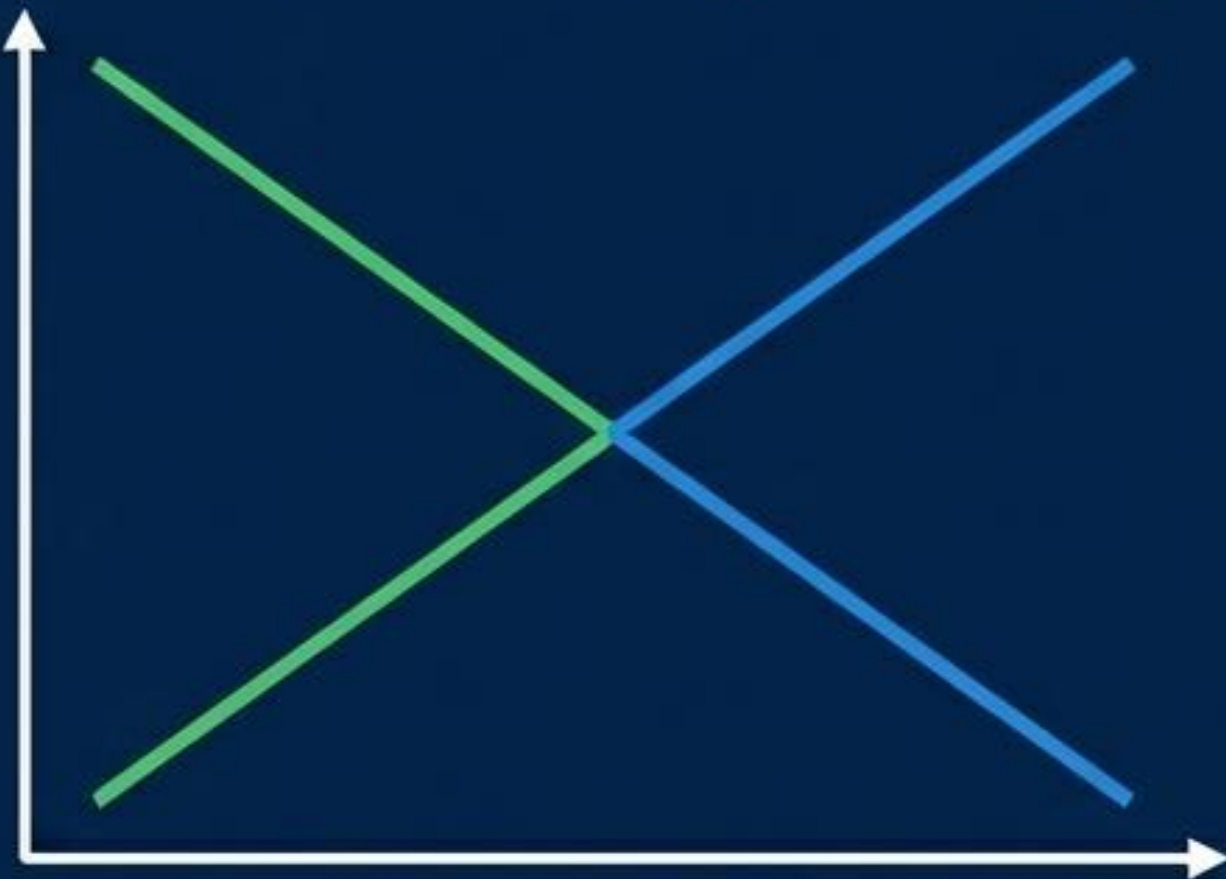
Ventajas:

- **50% Memoria** (Solo se guarda el triángulo inferior).
- **50% Operaciones** (Más rápido que LU).
- **Estable:** No requiere pivoteo.

Diagnóstico de Sistemas: Condicionamiento y Sensibilidad

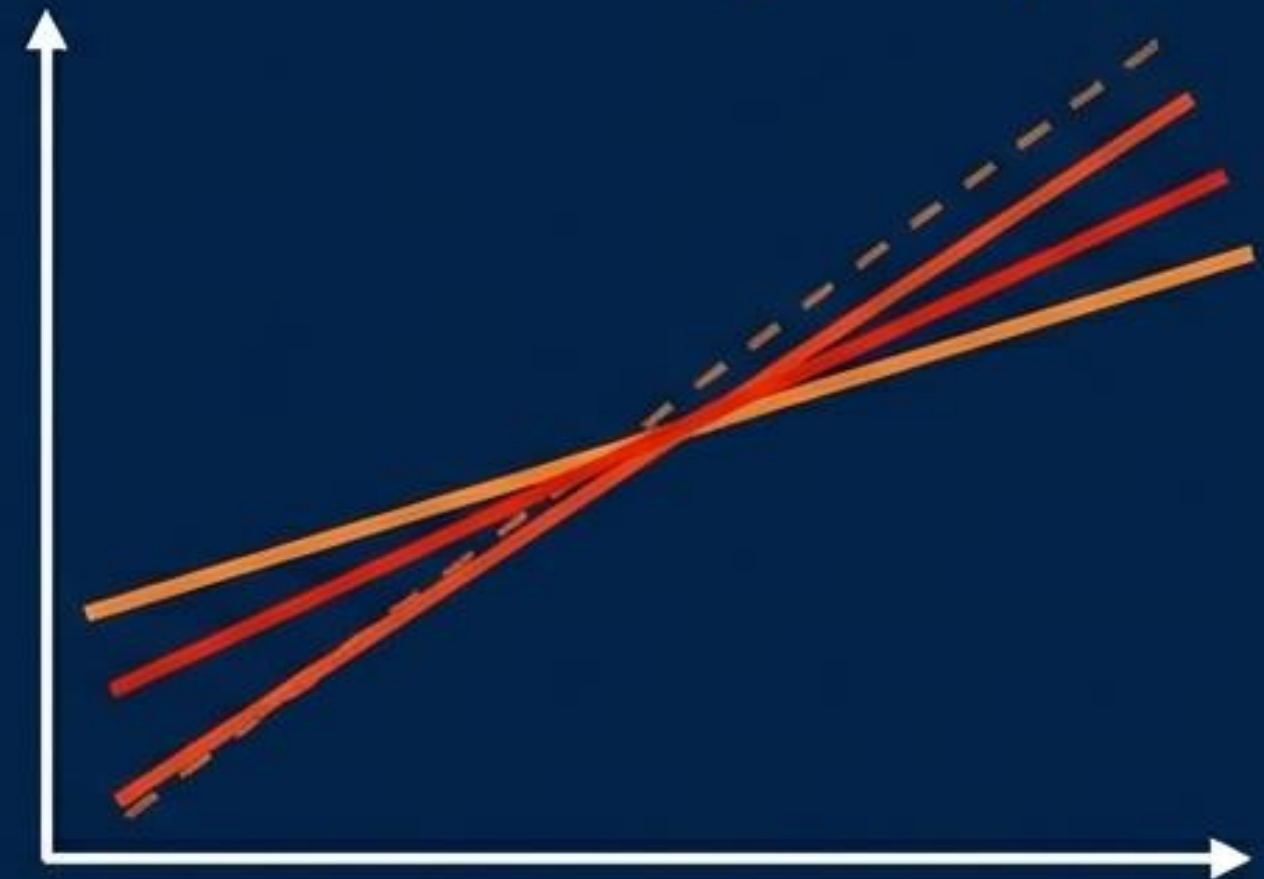
Número de Condicionamiento: $\text{cond}(A) = \|A\| \cdot \|A^{-1}\|$

Bien Condicionado ($\text{cond}(A) \approx 1$)



Pequeño error en datos →
Pequeño error en solución.

Mal Condicionado ($\text{cond}(A) \gg 1$)



Pequeña perturbación en datos →
Cambio catastrófico en solución.

Aplicación en Ingeniería de Energía: Circuitos Resistivos

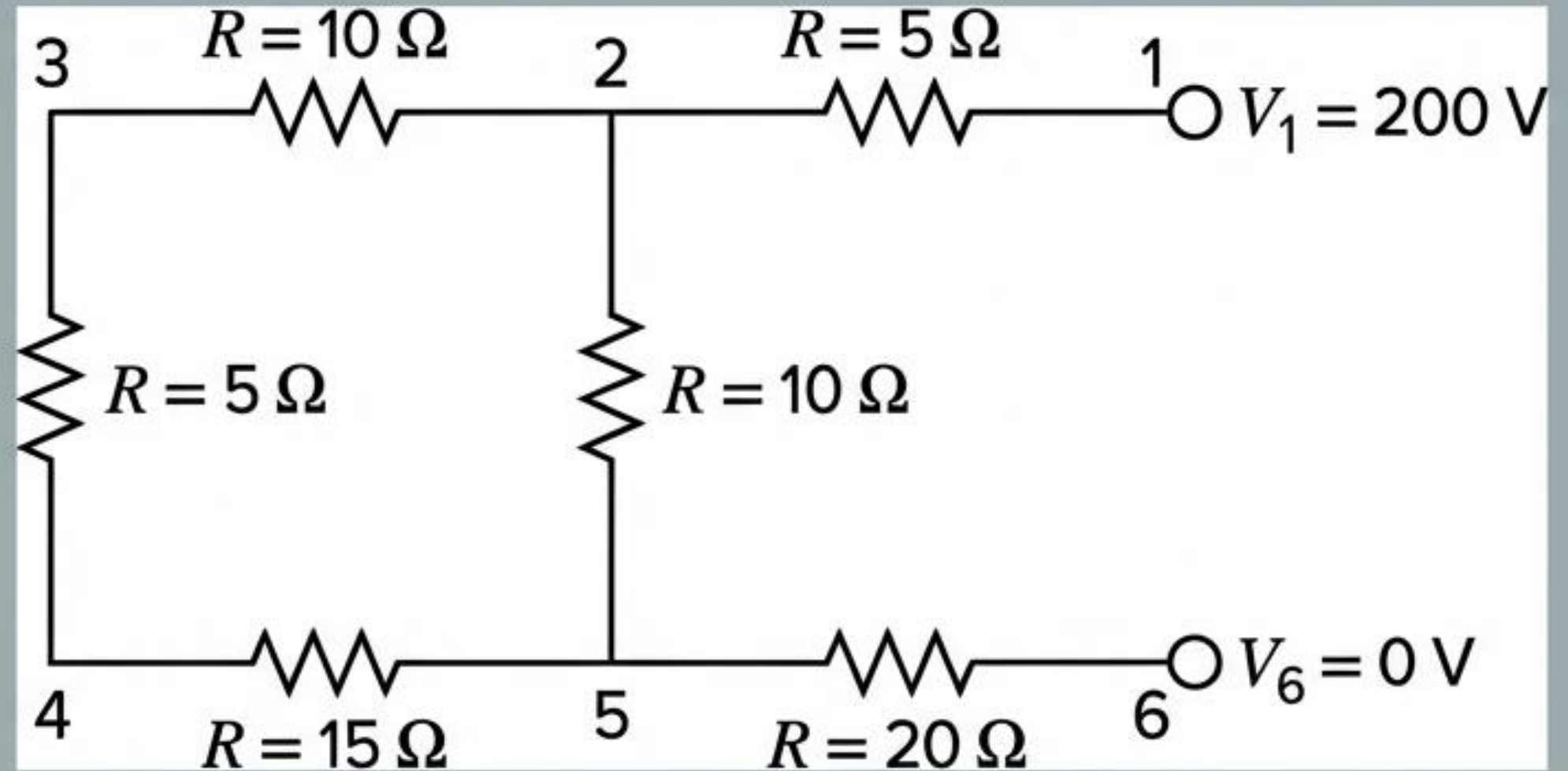
Contexto: Microrred Eléctrica

Principios Físicos:

1. Ley de Corrientes de Kirchhoff (LCK):

$$\sum I_{entrada} = \sum I_{salida}$$

2. Ley de Voltajes de Kirchhoff (LVK): $\sum V = 0$ en malla cerrada.



Modelado Matricial y Justificación de la Factorización LU

$$\underbrace{[R]}_{\text{Matriz de Resistencias}} \cdot \underbrace{[I]}_{\text{Vector de Corrientes}} = \underbrace{[V]}_{\text{Vector de Voltajes}}$$

Matriz de Resistencias
(Infraestructura fija, costosa de invertir)

Vector de Corrientes
(Incógnitas)

Vector de Voltajes
(Variable minuto a minuto)

¿Por qué LU?

- En una red real, la topología (R) es constante, pero la demanda (V) cambia.
- Solución: Factorizamos $R = LU$ UNA sola vez.
- Para cada nuevo V_{nuevo} , solo resolvemos sustituciones ($O(n^2)$) en lugar de Gauss completo ($O(n^3)$). Permite simulación en tiempo real.

Resumen Comparativo de Métodos Directos

Método	Características	Costo	Mejor Uso
Gauss Simple	Didáctico, inestable	$O(n^3)$	Aprendizaje académico
Gauss + Pivoteo ✓ Estándar de Ingeniería	Robusto, estándar	$O(n^3)$	Sistemas generales $Ax = b$
Factorización LU	Desacopla A de b	Factorización $O(n^3)$ + Solución $O(n^2)$	Múltiples vectores de carga b
Cholesky ✓ Estándar de Ingeniería	Requiere Simetría + Invariante	$O(n^3/6)$ (Muy rápido)	Elementos Finitos, Matrices de Rigidez

Actividad Práctica: El Copiloto Digital

El Reto

- **Tarea:** Optimización de Scripts y Diagnóstico.
- 1. **Prompt Inicial:** Solicitar a ChatGPT/Claude un script MATLAB para resolver sistema 10x10 usando Gauss simple.
- 2. **Modificación:** Solicitar la inclusión de 'Pivoteo Parcial' para manejar ceros.



> Genera un código MATLAB para resolver $Ax=b$ (10x10) sin funciones integradas.

```
% Script MATLAB para Gauss Simple 10x10 sin funciones integradas
A = rand(10, 10); % Matriz de coeficientes
b = rand(10, 1); % Vector de términos independientes
n = length(b);
Aug = [A, b]; % Matriz aumentada

% Eliminación hacia adelante
for k = 1:n-1
    for i = k+1:n
        m = Aug(i, k) / Aug(k, k);
        Aug(i, k:n+1) = Aug(i, k:n+1) - m * Aug(k, k:n+1);
    end
end

% Sustitución hacia atrás
x = zeros(n, 1);
x(n) = Aug(n, n+1) / Aug(n, n);
for i = n-1:-1:1
    x(i) = (Aug(i, n+1) - Aug(i, i+1:n) * x(i+1:n)) / Aug(i, i);
end

disp('Solución x:');
disp(x);
```


Análisis Crítico: Hombre vs. Máquina

Checklist de Validación Técnica



¿Omitió la Sustitución hacia atrás?

Las IAs frecuentemente triangularizan la matriz pero olvidan calcular el vector $\mathbf{x}$ final.



Manejo de Índices

Verificar si el bucle maneja índices de MATLAB (base 1) correctamente.



Validación de Estabilidad

¿El código verifica $\text{cond}(\mathbf{A})$? Si $\text{cond}(\mathbf{A})$ es alto (10^{14}), el código fallará silenciosamente.
Lección: La IA escribe sintaxis, el ingeniero garantiza integridad numérica.

Logros Alcanzados

- ✓ Representación algebraica y matricial de SEL.
- ✓ Clasificación de soluciones (Rouché–Frobenius).
- ✓ Implementación de Eliminación Gaussiana y Sustitución.
- ✓ Aplicación de Pivoteo Parcial para estabilidad.
- ✓ Factorización Avanzada (LU y Cholesky).
- ✓ Diagnóstico de Condicionamiento.

***“La aritmética exacta es una ilusión;
la estabilidad numérica es la meta.”***

Análisis Crítico: Hombre vs. Máquina

Checklist de Validación Técnica



¿Omitió la Sustitución hacia atrás?

Las IAs frecuentemente triangularizan la matriz pero olvidan calcular el vector $\mathbf{x}$ final.



Manejo de Índices

Verificar si el bucle maneja índices de MATLAB (base 1) correctamente.



Validación de Estabilidad

¿El código verifica $\text{cond}(\mathbf{A})$? Si $\text{cond}(\mathbf{A})$ es alto (10^{14}), el código fallará silenciosamente.
Lección: La IA escribe sintaxis, el ingeniero garantiza integridad numérica.